EXPERIMENT - 10

Customer Order Query Optimization

AIM

To generate a dataset containing at least 10,000 rows, identify five slow SQL queries using EXPLAIN and EXPLAIN ANALYZE, optimize them using indexing and query rewriting, and compare execution performance before and after optimization.

1. CREATE DATASET

```
CREATE DATABASE IF NOT EXISTS Experiment10;
USE Experiment10;
CREATE TABLE Customer_Order (
 Order_ID INT PRIMARY KEY, Customer_ID INT,
 Customer_Name VARCHAR(60), City VARCHAR(50),
 Product_Category VARCHAR(40), Product_Name VARCHAR(80),
 Quantity INT, Unit_Price DECIMAL(10,2), Order_Date DATE,
 Payment_Mode VARCHAR(20), Order_Status VARCHAR(20),
 Total_Amount DECIMAL(12,2)
) ENGINE=InnoDB;
SET SESSION cte_max_recursion_depth=11000;
INSERT INTO Customer_Order (...)
WITH RECURSIVE seq AS (
 SELECT 1 AS n UNION ALL SELECT n+1 FROM seq WHERE n<10000
)
SELECT n, 1000+MOD(n,1000),
 CONCAT('Customer_',1000+MOD(n,1000)),
 CASE MOD(n,5) WHEN 0 THEN 'Delhi' WHEN 1 THEN 'Lucknow'
 WHEN 2 THEN 'Kanpur' WHEN 3 THEN 'Noida' ELSE 'Jaipur' END,
 CASE MOD(n,4) WHEN 0 THEN 'Electronics' WHEN 1 THEN 'Grocery'
 WHEN 2 THEN 'Clothing' ELSE 'Books' END,
 CONCAT('Product_',MOD(n,100)), 1+MOD(n,5),
 200+MOD(n,50)*25,
 DATE_ADD('2026-01-01',INTERVAL MOD(n,300) DAY),
 CASE MOD(n,3) WHEN 0 THEN 'UPI' WHEN 1 THEN 'CARD' ELSE 'CASH' END,
 CASE MOD(n,4) WHEN 0 THEN 'DELIVERED' WHEN 1 THEN 'PENDING'
 WHEN 2 THEN 'SHIPPED' ELSE 'CANCELLED' END,
 (1+MOD(n,5))*(200+MOD(n,50)*25)
FROM seq;
SELECT COUNT(*) AS Total_Rows, MIN(Order_Date) AS First_Order,
 MAX(Order_Date) AS Last_Order FROM Customer_Order;
```

OUTPUT: DATASET VERIFICATION

Total_Rows	First_Order	Last_Order
10000	2026-01-01	2026-10-27

2-3. CITY FILTER - BEFORE AND AFTER INDEX

```
EXPLAIN ANALYZE
SELECT * FROM Customer_Order WHERE City='Lucknow';
CREATE INDEX idx_city ON Customer_Order(City);
EXPLAIN ANALYZE
SELECT * FROM Customer_Order WHERE City='Lucknow';
```

OUTPUT

Compare the access plan, actual rows, and timing in the two EXPLAIN ANALYZE result trees. Measurements depend on the MySQL server.

4-5. CUSTOMER AND DATE FILTER

```
-- BEFORE
EXPLAIN ANALYZE
SELECT * FROM Customer_Order
WHERE Customer_ID=1250
AND Order_Date BETWEEN '2026-02-01' AND '2026-06-30';
-- COMPOSITE INDEX AND AFTER
CREATE INDEX idx_customer_date
ON Customer_Order(Customer_ID, Order_Date);
EXPLAIN ANALYZE
SELECT * FROM Customer_Order
WHERE Customer_ID=1250
AND Order_Date BETWEEN '2026-02-01' AND '2026-06-30';
```

OUTPUT

Index	Indexed columns
idx_customer_date	Customer_ID, Order_Date

6-7. FUNCTION ON DATE COLUMN AND QUERY REWRITE

```
-- BEFORE: function is applied to the column
EXPLAIN ANALYZE
SELECT COUNT(*) AS Total_Orders
FROM Customer_Order WHERE YEAR(Order_Date)=2026;
-- AFTER: use a date range predicate
EXPLAIN ANALYZE
SELECT COUNT(*) AS Total_Orders
FROM Customer_Order
WHERE Order_Date>='2026-01-01'
AND Order_Date<'2027-01-01';
```

OUTPUT

Before	After
YEAR(Order_Date)=2026	Order_Date >= 2026-01-01 and < 2027-01-01

The range rewrite avoids applying YEAR() to the column. The source does not create an Order_Date index, so the rewritten query may still scan rows.

8-9. CATEGORY AND STATUS FILTER

```
-- BEFORE
EXPLAIN ANALYZE
SELECT * FROM Customer_Order
WHERE Product_Category='Electronics'
AND Order_Status='DELIVERED';
-- COMPOSITE INDEX AND AFTER
CREATE INDEX idx_category_status
ON Customer_Order(Product_Category, Order_Status);
EXPLAIN ANALYZE
SELECT * FROM Customer_Order
WHERE Product_Category='Electronics'
AND Order_Status='DELIVERED';
```

OUTPUT

Index	Indexed columns
idx_category_status	Product_Category, Order_Status

10-11. CITY AGGREGATION

```
-- BEFORE
EXPLAIN ANALYZE
SELECT City, SUM(Total_Amount) AS Total_Sales
FROM Customer_Order GROUP BY City
ORDER BY Total_Sales DESC;
-- COVERING INDEX AND AFTER
CREATE INDEX idx_city_total
ON Customer_Order(City, Total_Amount);
EXPLAIN ANALYZE
SELECT City, SUM(Total_Amount) AS Total_Sales
FROM Customer_Order GROUP BY City
ORDER BY Total_Sales DESC;
```

OUTPUT

Index	Indexed columns
idx_city_total	City, Total_Amount

12. FINAL INDEX VERIFICATION

```
SHOW INDEX FROM Customer_Order;
```

Index name	Columns
PRIMARY	Order_ID
idx_city	City
idx_customer_date	Customer_ID, Order_Date
idx_category_status	Product_Category, Order_Status
idx_city_total	City, Total_Amount

13. BEFORE AND AFTER PERFORMANCE COMPARISON

Query	Before optimization	After optimization
Query 1 - City Filter	no index on City	idx_city
Query 2 - Customer + Date	no composite index	idx_customer_date
Query 3 - YEAR(Order_Date)	function on column	date range rewrite
Query 4 - Category + Status	no composite index	idx_category_status
Query 5 - City Aggregation	no covering index	idx_city_total

RESULT

The generated dataset contains 10,000 rows. The five query pairs show the supplied index and rewrite changes. Record actual timings and access paths from EXPLAIN ANALYZE on the target MySQL server; the date-range rewrite is not paired with a standalone Order_Date index in the supplied experiment.